

Supervised Learning

Abstract—Supervised learning is a subclass of machine learning algorithms that aims to learn functions that map inputs to outputs. In this project we aim to analyze the performance of five different types of supervised learning algorithms, on two separate classification datasets.

I. INTRODUCTION

Supervised learning is a powerful machine learning technique that has applications to many fields. In recent years it has become so pervasive due to the increase in computational power of computers, as well as the availability of easy-to-use libraries such as Scikit-Learn, TensorFlow, PyTorch, etc... However, the unfortunate consequence of this is that many developers will simply “throw” a wide variety of supervised learning algorithms at a problem without a complete understanding of how the algorithms work and how to interpret their results. This lack of fundamental understanding often leads to the inability of a developer to efficiently tune algorithms effectively and to concisely interpret results. This can lead to wasted development time, uninterpretable results, and most importantly, models that are not robust to unseen data. With this in mind, the purpose of this project is to understand supervised learning at a deeper level. In order to accomplish this, we will “implement” 5 different supervised learning algorithms - Decision Trees, Neural Networks, AdaBoost, Support Vector Machines, K-Nearest Neighbors - and apply these algorithms to two separate datasets. Specifically, I have chosen two, interesting binary classification datasets that will be described further in the following section. I will perform an in-depth analysis on how these algorithms perform on these two datasets, comparing and contrasting both at an inter-algorithm as well as an inter-dataset level.

II. DATASETS

Dataset 1 - Predict 5-Year Longevity for NBA

Predicting longevity in professional sports is incredibly important for modern teams so that they can more efficiently allocate their resources to players who will benefit them in the future. This binary classification dataset consists of statistics from the rookie seasons of 1,318 NBA (National Basketball Association) players and a binary label indicating whether they played 5 or more years in the NBA. There are 19 feature variables, all of which are numerical statistics. The dataset is moderately imbalanced, consisting of about 62% positive labels. What further makes this dataset interesting is that this dataset only contains statistical features, leaving out many other potentially

important features of a player with high predictive value, such as height, weight, injury history etc.. This could lead to situations in which two instances (players) which have similar features (statistics) in their rookie seasons, but have different target labels, potentially due to one sustaining a serious injury, or other confounding factors. Having a dataset where there are instances with identical features but have different outputs could potentially cause performance issues with supervised learning algorithms. A 70%-30%, stratified train-test split was performed on the dataset in order to ensure that the training and testing sets both contain a similar proportion of positive labels.

Dataset 2 - Credit Card Fraud

As use of credit cards has become more pervasive, so has credit card fraud. Even isolated instances of credit card fraud could cause substantial financial losses for credit card companies, businesses, and individual consumers. Nonetheless, the proportion of credit card transactions that are fraud is relatively microscopic compared to non-fraudulent transactions. This binary classification, credit card fraud dataset consists of 25,134 instances, each containing 14 features. Unlike the NBA dataset which only consisted of continuous features, this dataset consists of categorical features such as “Income Type”, binary features such as “Gender”, as well as continuous variable such as “Income”. This dataset is also significantly more unbalanced than the NBA dataset, consisting of only 1.7% instances positive for fraud. For this dataset, any categorical feature with no natural ordering was removed, while those with a natural ordering were converted to integers. Similarly to the NBA datasets, a 70%-30%, stratified train test split was performed on the dataset in order to ensure that the training and testing sets both contain a similar proportion of positive labels. Furthermore, after noticing that the algorithms were not sufficiently learning a proper representation of the positive class due to the severe imbalance, I decided to under-sample the negative class of the training dataset so that both classes were equally distributed. This obviously has the disadvantage of throwing out possibly useful instances from the negative class, however, for simplicity and time constraints, this method will allow the supervised learning models to learn a more balanced representation of the two classes. I left the testing set imbalanced in order to achieve an unbiased evaluation during testing and to avoid data leakage.

Evaluation Metric - F1 Score

Since both datasets are imbalanced, binary classification datasets (with the fraud dataset being severely imbalanced) using a simple evaluation metric like “accuracy” will not suffice in providing an accurate view of performance. This is because we would be able

to achieve almost a 99% accuracy on the fraud dataset by simply predicting non-fraudulent for all instances. Obviously this would lead to no fraud detected which is not ideal. Therefore, in order to best quantify performance I have chosen to use the F1-Score evaluation metric. This metric combines both the “precision” and “recall” metric using a harmonic mean. Together, both of these metrics quantify both types of errors that could occur in classification. Precision quantifies the proportion of false positives and recall quantifies the proportion of false negatives. Any sort of binary classification problem involves a delicate balancing act between both types of errors, since a model with high recall might flag all positive example but potentially could flag too many, while a model with high precision might correctly classify all positive examples but simply because it is not flagging many to begin with.

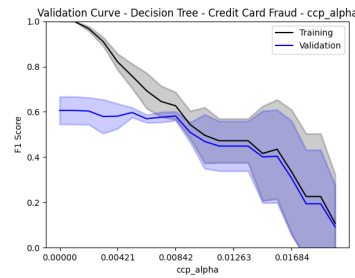
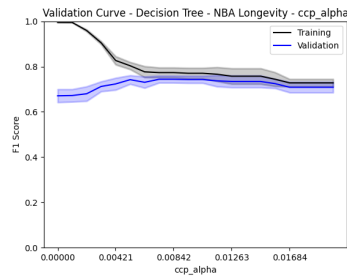
III. ANALYSIS

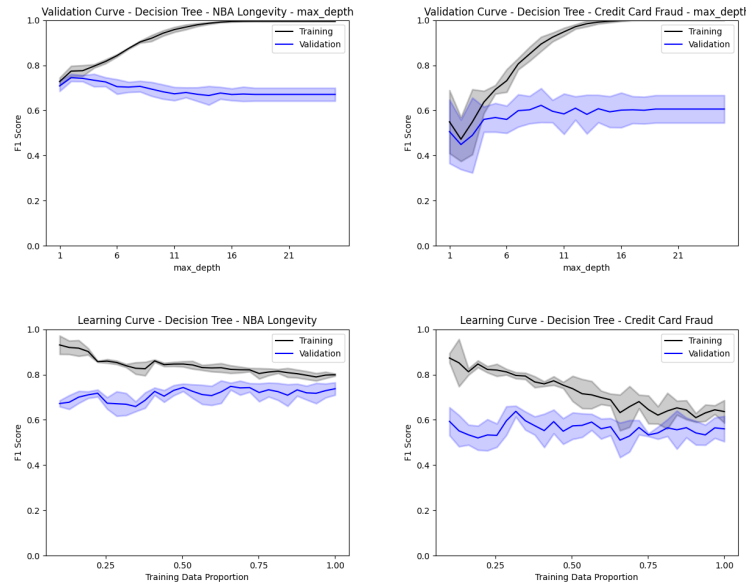
Methods

For each supervised learning algorithm, and for each dataset, I have generated validation curves for each hyper-parameter, as well as learning curves. For each validation curve and learning curve, I ran 5-fold cross validation on the training datasets, and each plot shows the mean f1-scores as well as the standard deviation across the 5 folds.

i) Decision Trees

The decision tree hyper-parameters that I chose to analyze were the minimal cost-complexity pruning parameter (ccp_alpha) and the maximum depth of the tree (max_depth). I chose to use Gini Impurity as the measurement in which to decide which features to split on. Gini Impurity is less computationally expensive than other splitting methods such as entropy, yet still provides a simple way to quantify the class purity of potential splits.





Minimal Cost-Complexity Pruning

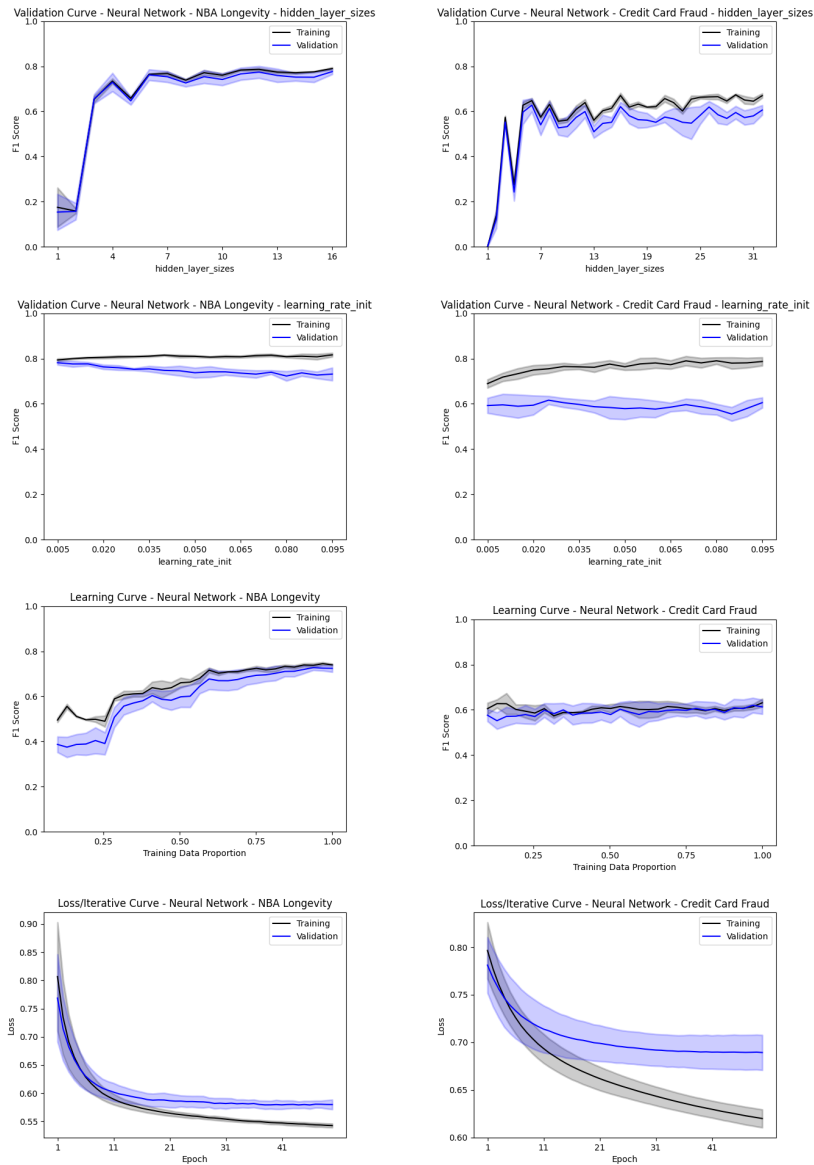
Minimal Cost-Complexity Pruning aims to mitigate overfitting by pruning nodes in the decision tree, with higher `ccp_alpha` values favoring smaller trees. In the plots above we can see that for both datasets, as `ccp_alpha` increases, the training score decreases. This is due to the trees being pruned more aggressively leading to less training instances being classified correctly (underfitting). For the NBA dataset, the validation score initially begins to rise as `ccp_alpha` increases. This is because when `ccp_alpha` is 0, the decision tree is overfitting, and as more nodes are pruned, less overfitting occurs. However, we notice after a certain point, validation score decreases again due to underfitting (too many nodes being pruned). The fraud validation curve appears to achieve best validation scores with less pruning, possibly due to noise in the dataset.

Max Depth

The `max_depth` parameter determines the maximum depth allowed for the decision tree. Larger trees are more prone to overfitting, so tuning this parameter aims to find the sweet spot between under and overfitting. The NBA dataset achieves its maximum validation score with a maximum depth of either 2 or 3. With depths any larger, the validation score begins to decrease due to the increase in the tree complexity. On the other hand, the fraud dataset appears to achieve better validation score with larger trees. This is potentially due to similar feature distributions between both classes, or the inability of under-sampling to compute a robust sampling of the negative class. I think the performance on fraud dataset could be improved with a more robust up-sampling method for the positive class. This way, too many negative instances would not need to be removed.

ii) Neural Networks

The neural network hyper-parameters that I chose to analyze were the number of nodes per layer and the learning rate. After some primitive tuning of the many other hyper-parameters that neural networks have, I decided to keep as a constant among all experiments, 1 hidden layer, relu activation functions on the hidden layer, and 50 epochs/iterations. I chose 1 hidden layer because I noticed that the more hidden layers there were, the more severe overfitting occurred, and I also noticed that after 50 epochs, there was not much further benefit for the loss and f1 score. The loss function used was Binary Cross Entropy Loss since it quantifies loss for binary classification problems.



Number of Nodes per Hidden Layer

The number of hidden nodes in a neural network decides how many connections there are from one layer to the next. The more nodes/connections between layers there are, then the more parameters need to be learned. In the plots for each dataset, one can appreciate how if the number of nodes is too small, then the model does not have enough parameters to learn a proper representation of even the training set (under-fitting). On the other hand, when the number of nodes is too great, then the model can overfit, which is seen in the fraud plot. The training score continues to increase while the validation score begins to decrease. The NBA fraud plot seems to have very little difference between the training and validation score. It's likely that if I trained the network for many more epochs, we would see the validation score decrease due to overfitting.

Learning Rate

When training a neural network, the learning rate is a factor that controls how large the weights/parameters are moved in the direction of the gradients. Too large a learning rate and the weights may move too much and “jump” over the optimal minimum solution, but too small a learning rate and it may take an unreasonable amount of epochs/iterations to reach a minimum in the loss function. In both the NBA and fraud plots, as the learning rate increase, the f1 score decreases. This is most likely due the weights being modified too much, and reaching a suboptimal solution. If I had more time I would've liked to explore using an adaptive learning rate.

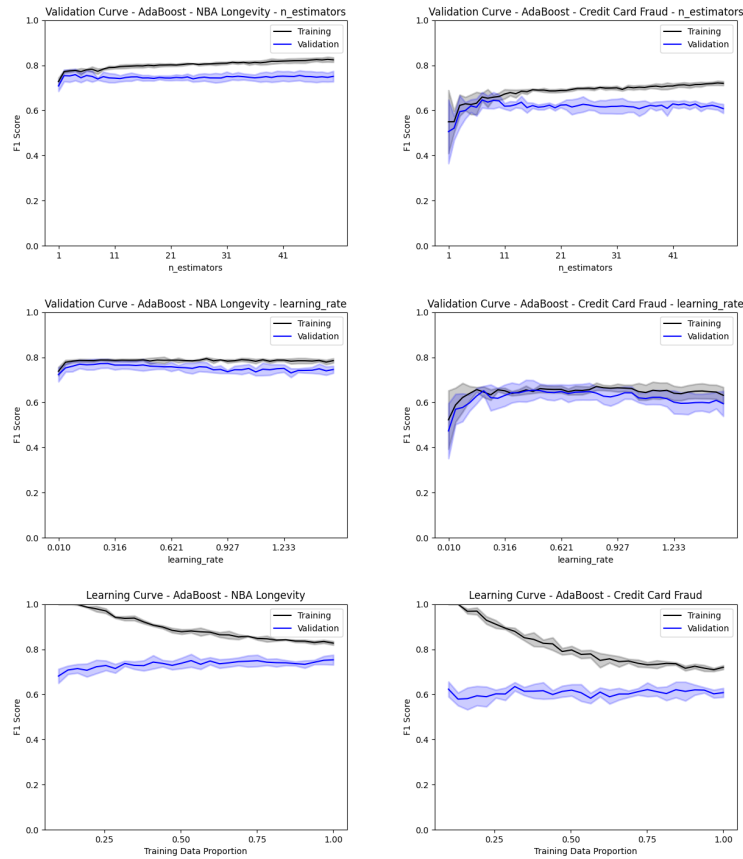
Learning Curves and Iterative Curves

In general, the more data a neural network sees during the training, the better it will be able to generalize to the validation dataset. This is evident in both learning curves, however, in the fraud plot, this increase in performance is marginal. This is most likely due to uninformative features in this dataset. We can also see in the iterative curves how the training losses decrease without bound, while the validation losses begin to stabilize. This is the point at which the neural networks begin to overfit. This overfitting seems to be more severe for the fraud dataset, most likely due to the size of the dataset being much smaller than the NBA dataset. With a relatively small number of parameters, it is much easier for a neural network to quickly overfit on smaller datasets.

iii) AdaBoost

The AdaBoost hyper-parameters that I chose to analyze were the maximum number of estimators at which boosting is terminated (`n_estimators`) and the weight applied to each classifier at each

boosting iteration (learning_rate). The base estimator that I chose was decision trees, since it would be interesting to compare AdaBoost using decision trees with just decision trees which we analyzed earlier in the report.



Number of Estimators

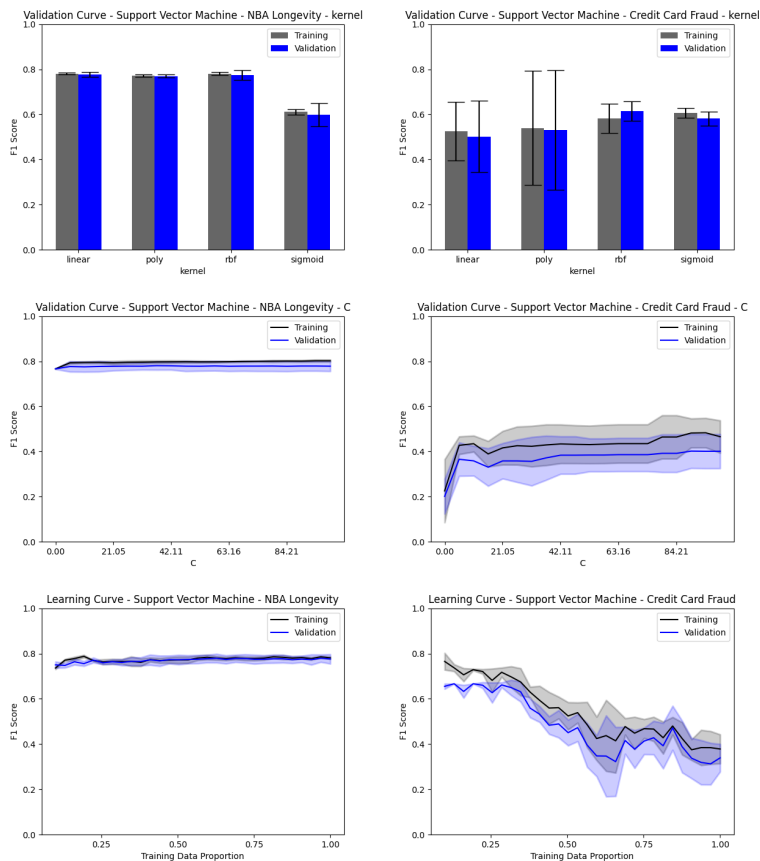
The number of estimators determines how many estimators (decision tree stumps in this case) are trained and used for prediction. Too many estimators or weak learners cause overfitting, which is evident in the above plots. For the NBA plots once can see the maximum validation score occurs with around 2 to 5 estimators, and then begins to drop off. The scores for the fraud dataset are fairly low throughout, due the data most likely containing noise outliers. The more noisy outliers there are, the more difficulty AdaBoost will have converging, since it will always try to add more weight to the misclassified outliers.

Learning rate

The learning rate in AdaBoost determines the weight applied to each classifier. A higher learning rate will increase the contribution to each classifier and a trade-off exists between the learning rate and the number of estimators. This is because even if more weight is added to each classifier, if there are many estimators, then each of these weights will have a very minor effect on the final prediction. One can see how once again the NBA dataset significantly outperforms the fraud dataset, regardless of the learning rate.

iv) Support Vector Machines

The Support Vector Machine hyper-parameters that I chose to analyze were the kernel type and the regularization parameter (C). Here I plot the kernel type analysis as a base plot.



Kernel Type

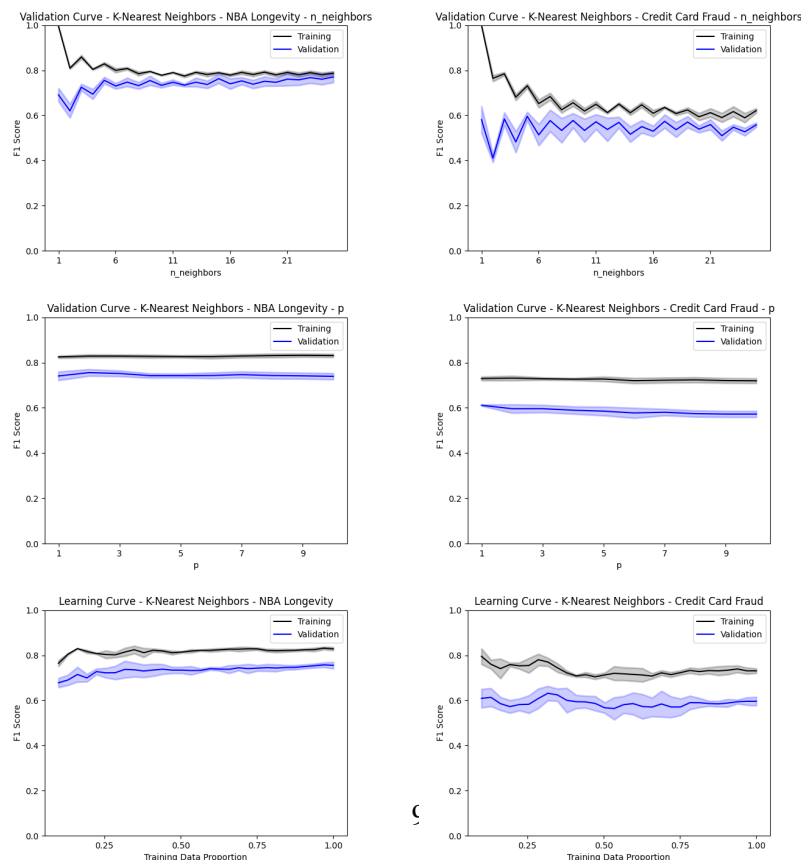
The purpose of kernels in Support Vector Machines is to transform the data to a higher/different space in which the data may be linearly separable. One can see that across both datasets, that the kernel with best validation performance is the radial basis (rbf) kernel. The rbf is one of the most commonly used kernels since it is technically an infinite dimension kernel. It is also evident from the above plots that the fraud database performed significantly worse than the NBA dataset regardless of the kernel and had much higher variance between folds. Once again this can most likely be contributed to noisy and imbalanced data.

Regularization Parameter (C)

The regularization parameter controls the strength of L2 regularization on the SVM weights. Here we can see a significant difference between the fraud and NBA datasets. The NBA datasets seems to converge to decent performance regardless of the value of C, while the fraud dataset achieves performance no better than 0.5.

v) K-Nearest Neighbors

The K-Nearest Neighbors hyper-parameters that I chose to analyze were the number of neighbors (n_neighbors) and the power parameter (p) for the Minkowski metric.



Number of Neighbors (K)

The K parameter in K-Nearest Neighbors determines the number of points to take into consideration surrounding a query point. When K is too small, this can cause overfitting since the each query point is simply relying on its closest neighbors. However, when K is too large this can cause under-fitting since each query point will be considering many points around it, even once that are not close. We can see clearly from the plots above how the difference in each datasets is highlighted by K. The validation score for the NBA dataset continues to increase as K increases, while the score for the fraud datasets does not benefit from increasing values of K.

Power Parameter for Minkowski Metric

The power parameter (p) for the Minkowski metric determine the exact distance metric equation being used. When $p=1$, the distance metric become the straight line Manhattan distance and when $p=2$ the metric becomes Euclidean distance. As p increases, the distances between all points becomes very similar. It is not necessarily clear from my plots if p had any significant effect on the result of this algorithm

IV. Results

After determining the best hyper-parameters via the previous analyses, the models were trained once again with the entire training set, and then the model were used to predict on the unseen testing set. The result are summarized in the following table along with time to train.

NBA Dataset	Hyper-Parameters	Time to Train (s)	F1-Score
Decision Tree	max depth = 3 ccp_alpha = .008 criterion = Gini Impurity	0.004	0.736
Neural Network	# hidden layers = 1 Hidden layer size = 10 Learning rate = .005 Max iterations = 50	0.088	0.763
AdaBoost	Base estimator = DTree # estimators = 3 Learning rate = .25	0.01	0.743
SVM	Kernel = rbf C = 1.5	0.037	0.777

NBA Dataset	Hyper-Parameters	Time to Train (s)	F1-Score
KNN	# Neighbors = 6 p = 2	0.001	0.736

Fraud	Hyper-Parameters	Time to Train (s)	F1-Score
Decision Tree	max depth = 3 ccp_alpha = .008 criterion = Gini Impurity	0.002	0.058
Neural Network	# hidden layers = 1 Hidden layer size = 10 Learning rate = .02 Max iterations = 50	0.013	0.033
AdaBoost	Base estimator = DTree # estimators = 10 Learning rate = .6	0.013	0.05
SVM	Kernel = rbf C = 10	0.02	0.028
KNN	# Neighbors = 6 p = 2	0.007	0.047

V. Discussion of Results

For the NBA dataset, all of the final models performed rather similarly. The best model turned out to be the Support Vector Machine, but only by a small margin. This is likely due to the radial basis kernel being able to transform the data into an ‘infinite dimensional’ space. The quickest model to train was K-Nearest Neighbors. This makes sense since there is no training computation done really. All computation comes during query time. Overall, a final F1-score of .777 is fairly decent for this problem, considering the minor class imbalance.

In contrast, the fraud dataset performed very poorly on the testing set. Far worse than was expected, since the cross validation for most models achieved about a .5 F1-score. This is due I believe to the severe class imbalance, and also due to the methods in which I tried to mitigate the class imbalance with. By under-sampling the training set, I removed a huge proportion of the data, and in retrospect this was probably not a great idea. While performing cross validation on the training set, which had 50% positive examples, achieving a .5 F1-Score was most likely due to chance, and the models were essentially guessing. This led me to believe that the models were “learning”, when in fact they most likely were not learning any sort of meaningful representation

of the data. Another factor that could have contributed to these results was my pre-processing. I converted many of the nominal features into integers, assuming natural ordering for many of them. These assumption I have made may not have been correct. It would be interesting to see how the results would differ if I one hot encoded the nominal feature variables, however, this analysis would not fit in the scope of this project.

In retrospect, it was likely too ambitious of me to choose such an imbalanced dataset without having great knowledge of how to deal with such factors. If I had more time I would have liked to experiment with some different data imbalance techniques to see if they could improve performance. Overall though it was very interesting and insightful to see how these 5 algorithms would perform on different datasets. I believe these two datasets really did a great job of highlighting how these algorithms perform on different datasets, and gave me some deeper insight into which models may or may not work well on datasets with certain characteristics.

VI. REFERENCES

1. [Scikit-learn: Machine Learning in Python](#), Pedregosa *et al.*, JMLR 12, pp. 2825-2830, 2011.
2. Imbalanced Datasets, <https://www.kdnuggets.com/2017/06/7-techniques-handle-imbalanced-data.html>
3. Keras, <https://keras.io/>